# Sparse 3D Reconstruction: A Two-View Framework using AGAST, FREAK, and Levenberg-Marquardt Bundle Adjustment

Computer Vision Study Guide

## 1 Introduction to Two-View Structure from Motion

Structure from Motion (SfM) is the process of estimating 3D structures from a series of 2D images. In a two-view setup, the objective is to reconstruct a sparse 3D point cloud by establishing geometric correspondences between a pair of images. This pipeline strictly leverages a feature-based approach, relying on epipolar geometry, algebraic triangulation, and non-linear optimization to minimize reprojection errors.

## 2 Feature Extraction and Description

The foundation of any sparse reconstruction pipeline relies on finding highly distinctive points that can be reliably matched across varying viewpoints.

### 2.1 AGAST Corner Detector

The Adaptive and Generic Accelerated Segment Test (AGAST) is utilized to detect interest points. It improves upon the classic FAST detector by optimizing the decision trees used for corner detection, yielding higher performance without sacrificing repeatability. It evaluates a circular ring of pixels around a candidate pixel $p$, identifying a corner if a contiguous arc of pixels is significantly brighter or darker than the center.

### 2.2 FREAK Descriptor

Once keypoints are detected via AGAST, they must be described to allow for matching. Fast Retina Keypoint (FREAK) is a binary descriptor inspired by the human visual system, specifically the retina's topology. The sampling pattern consists of concentric circles with overlapping receptive fields, where the density of sampling points is higher near the center and decreases outward. This mimics the foveal structure of the eye. Because FREAK generates binary strings, matches are calculated using the Hamming distance.

## 3 Epipolar Geometry and Pose Estimation

### 3.1 The Essential Matrix

For two calibrated cameras, the relationship between corresponding points in the first image ($x_1$) and the second image ($x_2$) is governed by the Epipolar Constraint:

$$x_2^T E x_1 = 0 \tag{1}$$

where $E$ is the Essential Matrix, and $x_1, x_2$ are normalized image coordinates ($x = K^{-1}u$, with $u$ being the pixel coordinate and $K$ the intrinsic matrix). The Essential Matrix encodes the rigid body transformation (rotation $R$ and translation $t$) between the two camera centers:

$$E = [t]_\times R \tag{2}$$

where $[t]_\times$ is the skew-symmetric matrix of the translation vector.

### 3.2 Robust Estimation via RANSAC

Because feature matching is imperfect, the initial set of correspondences will contain outliers. Random Sample Consensus (RANSAC) is employed alongside the five-point or eight-point algorithm to estimate $E$. RANSAC iteratively selects minimal subsets of points to hypothesize models, eventually identifying the model with the largest consensus set (inliers).

### 3.3 Pose Recovery

Decomposing $E$ yields four possible mathematical solutions for $(R, t)$. Only one configuration places the reconstructed 3D points in front of both cameras. This is resolved via the chirality check, which tests the triangulated depth of points to ensure positive $Z$ values in both camera coordinate frames.

## 4 Linear Triangulation

With the relative pose established, the next step is to estimate the 3D position $X$ of the matched 2D points. The projection of a 3D point $X$ onto an image plane is defined by:

$$\lambda x = K[R|t]X = PX \tag{3}$$

where $\lambda$ is the depth, and $P$ is the camera projection matrix. By representing the cross product, $x \times PX = 0$, we can formulate a system of linear equations $AX = 0$ for a given pair of corresponding points. This homogeneous system is typically solved using Singular Value Decomposition (SVD), where the solution for $X$ corresponds to the right singular vector associated with the smallest singular value.

## 5 Bundle Adjustment: Non-Linear Optimization

Algebraic triangulation minimizes an algebraic error, which lacks geometric or physical meaning. Noise in feature extraction and quantization leads to inaccuracies. To refine the 3D points and camera parameters, we minimize the reprojection error: the geometric distance between the observed 2D feature and the projection of the estimated 3D point.

### 5.1 Levenberg-Marquardt Algorithm

The objective function to minimize is:

$$E(R, t, X) = \sum_i \left( \|u_{1,i} - \pi(K, I, 0, X_i)\|^2 + \|u_{2,i} - \pi(K, R, t, X_i)\|^2 \right) \tag{4}$$

where $\pi$ is the projection function. To solve this non-linear least-squares problem, the Levenberg-Marquardt (LM) algorithm is selected. LM is a hybrid technique that interpolates between the Gauss-Newton algorithm and the method of gradient descent.

The parameter update $\Delta\theta$ is solved via the normal equations:

$$(J^T J + \lambda I)\Delta\theta = J^T \mathbf{e} \tag{5}$$

where $J$ is the Jacobian matrix of the projection function with respect to the parameters, $\mathbf{e}$ is the residual vector, and $\lambda$ is the damping factor. When $\lambda$ is large, it behaves like gradient descent; when $\lambda$ is small, it mirrors Gauss-Newton, leading to rapid quadratic convergence near the local minimum.

## 6 Python Implementation

Below is the complete implementation of the theoretical pipeline using OpenCV and SciPy. The code demonstrates the extraction, matching, estimation, triangulation, and optimization phases.

```python
import cv2
import numpy as np
from scipy.optimize import least_squares
import matplotlib.pyplot as plt

def extract_and_match(img1, img2):
    detector = cv2.AgastFeatureDetector_create()
    descriptor = cv2.xfeatures2d.FREAK_create()

    kp1 = detector.detect(img1, None)
    kp2 = detector.detect(img2, None)

    kp1, des1 = descriptor.compute(img1, kp1)
    kp2, des2 = descriptor.compute(img2, kp2)

    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
    matches = bf.match(des1, des2)
    matches = sorted(matches, key=lambda x: x.distance)

    pts1 = np.float32([kp1[m.queryIdx].pt for m in matches])
    pts2 = np.float32([kp2[m.trainIdx].pt for m in matches])

    return pts1, pts2

def project_points(points_3d, rvec, tvec, K):
    points_2d, _ = cv2.projectPoints(points_3d, rvec, tvec, K, None)
    return points_2d.reshape(-1, 2)

def reprojection_residuals(params, n_points, pts1, pts2, K):
    rvec = params[:3]
    tvec = params[3:6]
    points_3d = params[6:].reshape((n_points, 3))

    proj1 = project_points(points_3d, np.zeros(3), np.zeros(3), K)
    proj2 = project_points(points_3d, rvec, tvec, K)

    error1 = (proj1 - pts1).ravel()
    error2 = (proj2 - pts2).ravel()

    return np.hstack((error1, error2))

def optimize_bundle(pts1, pts2, points_3d, R, t, K):
    rvec, _ = cv2.Rodrigues(R)
    initial_params = np.hstack((rvec.ravel(), t.ravel(), points_3d.ravel()))
    n_points = points_3d.shape[0]

    res = least_squares(
        reprojection_residuals,
        initial_params,
        method='lm',
        args=(n_points, pts1, pts2, K)
    )

    optimized_params = res.x
    opt_points_3d = optimized_params[6:].reshape((n_points, 3))

    return opt_points_3d
```

```
58
59 def execute_reconstruction(img_path1, img_path2, K):
60     img1 = cv2.imread(img_path1, cv2.IMREAD_GRAYSCALE)
61     img2 = cv2.imread(img_path2, cv2.IMREAD_GRAYSCALE)
62
63     pts1, pts2 = extract_and_match(img1, img2)
64
65     E, mask = cv2.findEssentialMat(pts1, pts2, K, cv2.RANSAC, 0.999, 1.0)
66     pts1_inliers = pts1[mask.ravel() == 1]
67     pts2_inliers = pts2[mask.ravel() == 1]
68
69     _, R, t, _ = cv2.recoverPose(E, pts1_inliers, pts2_inliers, K)
70
71     P1 = K @ np.hstack((np.eye(3), np.zeros((3, 1))))
72     P2 = K @ np.hstack((R, t))
73
74     points_4d = cv2.triangulatePoints(P1, P2, pts1_inliers.T, pts2_inliers.T)
75     points_3d = (points_4d[:3, :] / points_4d[3, :]).T
76
77     optimized_3d = optimize_bundle(pts1_inliers, pts2_inliers, points_3d, R, t,
       K)
78
79     fig = plt.figure()
80     ax = fig.add_subplot(111, projection='3d')
81     ax.scatter(optimized_3d[:, 0], optimized_3d[:, 1], optimized_3d[:, 2], s=2,
       c='r', marker='.')
82     plt.show()
```